

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЧЕРКАСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ БОГДАНА ХМЕЛЬНИЦЬКОГО**

**Факультет обчислювальної техніки, інтелектуальних та управляючих систем
Кафедра інформаційних технологій**

**ЗВІТ
з дисципліни: Проектування та архітектура ПЗ
з лабораторної роботи №2
Тема: Проектування прикладного програмного інтерфейсу
(API)**

Виконав:

студент групи КС-23 Тукало Іван Валерійович

Перевірив:

Кандидат технічних наук Мисник Богдан Вікторович

Черкаси – 2026

Завдання:

Розробити REST API до системи, яку ви почали описувати на першій лабораторній роботі. Задokumentувати його у вигляді списку API endpoints для всіх наявних ресурсів. Для кожного з endpoint надати:

- приклади request/response body в форматі json
- статуси виконання успішних запитів
- статуси можливих помилок та зрозумілі повідомлення про причини виникнення помилок

Рекомендовано використовувати Swagger <https://editor-next.swagger.io/> для створення документації REST API або Postman <https://www.postman.com/>

Корисні посилання

– <https://medium.com/@yusufacarr18/whats-api-api-types-most-popular-api-services-rest-vs-soap-what-s-the-difference-1bd6a685afa1>

– Список кодів HTTP

https://upload.wikimedia.org/wikipedia/commons/0/09/%D0%A1%D0%BF%D0%B8%D1%81%D0%BE%D0%BA_%D0%BA%D0%BE%D0%B4%D1%96%D0%B2_%D1%81%D1%82%D0%B0%D0%BD%D1%83_HTTP.pdf

Варіант 2 - Автоматизація роботи бібліотекаря.

Порядок виконання:

Для виконання завдання та створення документації використано Swagger.

Код для Swagger:

```
openapi: 3.0.0
info:
  title: Hybrid Library Management API
  description: Розширений REST API для комплексної автоматизації роботи гібридної бібліотеки.
  version: 1.1.0
servers:
  - url: https://api.library-hybrid.com/v1
    description: Основний робочий сервер
paths:
  /api/books:
    get:
      summary: Пошук у каталозі
      description: Повертає список доступних книг із підтримкою фільтрації.
      parameters:
        - name: author
          in: query
          required: false
          schema:
            type: string
        - name: is_digitized
          in: query
          required: false
          schema:
            type: boolean
      responses:
        '200':
          description: Успішний пошук.
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Book'
    post:
      summary: Додавання нової книги
      description: Дозволяє бібліотекарю внести новий екземпляр до бази даних.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/BookInput'
      responses:
        '201':
```

```
    description: Книгу успішно додано.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Book'
  '409':
    description: Книга з такими параметрами вже існує.

/api/books/{book_id}:
  get:
    summary: Отримати деталі книги
    description: Повертає повну інформацію про конкретну книгу за її
ідентифікатором.
    parameters:
      - name: book_id
        in: path
        required: true
        schema:
          type: integer
    responses:
      '200':
        description: Дані знайдено.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Book'
      '404':
        description: Книгу не знайдено.
  put:
    summary: Редагувати інформацію про книгу
    description: Оновлення даних про існуючий екземпляр у базі.
    parameters:
      - name: book_id
        in: path
        required: true
        schema:
          type: integer
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/BookInput'
    responses:
      '200':
        description: Дані успішно оновлено.
  delete:
    summary: Видалити книгу з бази
    description: Повне видалення запису про книгу у випадку списання або
втрати.
    parameters:
```

```
- name: book_id
  in: path
  required: true
  schema:
    type: integer
responses:
  '204':
    description: Запис успішно видалено.

/api/books/{book_id}/content:
  get:
    summary: Читання відцифрованої книги
    description: Надає доступ до тексту книги безпосередньо на сайті.
    parameters:
      - name: book_id
        in: path
        required: true
        schema:
          type: integer
    responses:
      '200':
        description: Доступ до контенту надано.
      '403':
        description: Відмовлено в доступі або книга існує лише у фізичному
        варіанті.

/api/auth/login:
  post:
    summary: Авторизація користувача
    description: Вхід до системи за допомогою номера телефону та пароля.
    requestBody:
      required: true
      content:
        application/json:
          example:
            phone: "+380671112233"
            password: "securePassword123"
    responses:
      '200':
        description: Успішна авторизація. Повертає токен доступу.
        content:
          application/json:
            example:
              access_token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
      '401':
        description: Невірний логін або пароль.

/api/users:
  post:
    summary: Реєстрація нового користувача
    description: Створення профілю читачем самостійно або бібліотекарем.
```

```
requestBody:
  required: true
  content:
    application/json:
      example:
        full_name: "Олександр Петренко"
        phone: "+380671112233"
        verification_code: "7788"
  responses:
    '201':
      description: Профіль успішно створено.

/api/users/{user_id}:
  delete:
    summary: Видалення власного акаунта
    description: Читач може ініціювати видалення свого профілю за умови відсутності боргів.
    parameters:
      - name: user_id
        in: path
        required: true
        schema:
          type: integer
    responses:
      '204':
        description: Акаунт назавжди видалено.
      '400':
        description: Видалення неможливе через наявність активних бронювань.

/api/users/me/orders:
  get:
    summary: Перегляд історії своїх ордерів
    description: Отримання списку поточних бронювань та термінів їх повернення для авторизованого читача.
    responses:
      '200':
        description: Успішно повернуто список ордерів.
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Order'

/api/orders:
  post:
    summary: Створення бронювання або видача
    description: Дозволяє читачу забронювати книгу або бібліотекару зафіксувати фізичну видачу.
    requestBody:
      required: true
```

```

    content:
      application/json:
        example:
          user_id: 501
          book_id: 201
          duration_days: 7
    responses:
      '201':
        description: Ордер успішно створено.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Order'
      '422':
        description: Немає вільних примірників.

/api/orders/{order_id}:
  delete:
    summary: Скасування бронювання
    description: Авторизований читач скасовує своє бронювання до моменту
фізичного отримання книги.
    parameters:
      - name: order_id
        in: path
        required: true
        schema:
          type: integer
    responses:
      '204':
        description: Бронювання скасовано, екземпляр знову доступний.
  patch:
    summary: Фіксація повернення книги
    description: Бібліотекар закриває ордер після того, як читач фізично
повертає екземпляр.
    parameters:
      - name: order_id
        in: path
        required: true
        schema:
          type: integer
    requestBody:
      required: true
      content:
        application/json:
          example:
            status: "returned"
    responses:
      '200':
        description: Книгу успішно прийнято в базу.

components:

```

```
schemas:
  Book:
    type: object
    properties:
      id:
        type: integer
        example: 201
      title:
        type: string
        example: "Чистий код"
      authors:
        type: array
        items:
          type: string
          example: ["Роберт Мартін"]
      publisher:
        type: string
        example: "Фабула"
      year:
        type: integer
        example: 2019
      free_copies:
        type: integer
        example: 10
      booked_copies:
        type: integer
        example: 0
      is_digitized:
        type: boolean
        example: false

  BookInput:
    type: object
    required:
      - title
      - authors
      - free_copies
    properties:
      title:
        type: string
        example: "Чистий код"
      authors:
        type: array
        items:
          type: string
          example: ["Роберт Мартін"]
      publisher:
        type: string
        example: "Фабула"
      year:
        type: integer
```



```

        example: 2019
    free_copies:
        type: integer
        example: 10
    is_digitized:
        type: boolean
        example: false

Order:
  type: object
  properties:
    id:
      type: integer
      example: 10042
    user_id:
      type: integer
      example: 501
    book_id:
      type: integer
      example: 201
    issue_date:
      type: string
      format: date
      example: "2026-04-30"
    return_date:
      type: string
      format: date
      example: "2026-05-07"
    status:
      type: string
      example: "booked"

```

Опис спроектованих API Endpoints

Документація прикладного програмного інтерфейсу для системи автоматизації гібридної бібліотеки розпочинається з ресурсу книжкового каталогу, доступного за адресою GET /api/books. Цей метод підтримує фільтрацію за параметрами author та is_digitized для зручного пошуку потрібної літератури. У разі успішного виконання сервер повертає статус 200 OK із JSON-масивом, що містить повну інформацію про кожен екземпляр, включаючи кількість вільних та заброньованих копій для уникнення конфліктів при видачі.

JSON

```

[
  {
    "id": 201,

```

```
    "title": "Чистий код",
    "authors": [
        "Роберт Мартін"
    ],
    "publisher": "Фабула",
    "year": 2019,
    "free_copies": 10,
    "booked_copies": 0,
    "is_digitized": false
}
]
```

Основна логіка управління фондом реалізована через базові CRUD-операції. Для створення нового запису в базі даних бібліотекар надсилає запит POST /api/books, передаючи у тілі JSON-об'єкт із характеристиками видання. Успішна операція підтверджується статусом 201 Created, а при спробі створити дублікат повертається помилка 409 Conflict.

```
JSON
{
    "title": "Чистий код",
    "authors": [
        "Роберт Мартін"
    ],
    "publisher": "Фабула",
    "year": 2019,
    "free_copies": 10,
    "is_digitized": false
}
```

Деталі конкретної книги доступні через запит GET /api/books/{book_id}. Редагування існуючої інформації здійснюється за допомогою методу PUT /api/books/{book_id}, а повне видалення запису з каталогу ініціюється методом DELETE /api/books/{book_id}, який повертає статус 204 No Content. Для доступу до електронного архіву використовується спеціальний метод GET /api/books/{book_id}/content, який надає можливість читання відцифрованої книги безпосередньо на сайті при успішній авторизації зі статусом 200 OK або блокує запит помилкою 403 Forbidden.

Керування доступом розпочинається з авторизації через POST /api/auth/login, де користувач отримує токен доступу. Реєстрація нових

профілів читачів відбувається через метод POST /api/users. Система приймає тіло запиту з персональними даними та SMS-кодом верифікації для підтвердження номера телефону.

```
JSON

{
  "full_name": "Олександр Петренко",
  "phone": "+380671112233",
  "verification_code": "7788"
}
```

Після успішної реєстрації зі статусом 201 Created користувач отримує право керувати власним профілем. Читач може ініціювати стирання своїх даних через метод DELETE /api/users/{user_id}. Проте сервер обов'язково перевіряє наявність заборгованостей і може відхилити запит із помилкою 400 Bad Request, якщо у клієнта є активні бронювання. Авторизовані користувачі також можуть переглядати свою історію взаємодії з бібліотекою через метод GET /api/users/me/orders.

Фіксація обігу літератури відбувається за допомогою ресурсу POST /api/orders. Запит ініціюється для бронювання книги на сайті читачем або для фіксації фізичної видачі бібліотекарем, передаючи ідентифікатори користувача, книги та бажаний термін дії.

```
JSON

{
  "user_id": 501,
  "book_id": 201,
  "duration_days": 7
}
```

Система повертає статус 201 Created при успішному створенні транзакції або 422 Unprocessable Entity, якщо вільних примірників наразі немає. Скасування активного резерву до моменту фізичного отримання книги виконується методом DELETE /api/orders/{order_id}, що автоматично повертає екземпляр у вільний доступ зі статусом 204 No Content. Завершальним етапом обслуговування є метод PATCH /api/orders/{order_id}, за допомогою якого бібліотекар оновлює статус ордеру після того, як читач фізично повертає книгу. Ця операція підтверджується статусом 200 OK і знімає заборгованість з профілю клієнта.

Висновки:

Під час виконання лабораторної роботи було успішно закріплено практичні навички моделювання веб-API додатка на прикладі архітектурного стилю REST. Для розроблюваної системи «Автоматизація роботи бібліотекаря» було спроектовано комплексний прикладний програмний інтерфейс, що охоплює всі ключові бізнес-процеси: пошук локальної та відцифрованої літератури, управління книжковим фондом, реєстрацію профілів читачів та контроль за резервуванням і фізичним обігом книг.

У ході проєктування було визначено основні ресурси системи та побудовано логічну структуру URL-шляхів для взаємодії з ними. Для реалізації як базових операцій управління даними, так і специфічних бізнес-дій було застосовано стандартні HTTP-методи. Для кожного запиту розроблено чітку структуру вхідних та вихідних даних у форматі JSON. Окрему увагу було приділено налаштуванню коректної взаємодії між клієнтом та сервером: описано сценарії успішного виконання операцій та передбачено обробку виняткових ситуацій із поверненням відповідних статусів і повідомлень про помилки, що гарантує цілісність бібліотечного фонду.